A Hybrid Architecture Design for SM4 Algorithm Based on FPGA

Wei Wang *
School of electrical engineering
Tongling University
Tongling, China
weiwang@tlu.edu.cn

Junfan Wu
School of electrical engineering
Tongling University
Tongling, China
2209172027@mail.tlu.edu.cn

Feixiang Du
School of electrical engineering
Tongling University
Tongling, China
feixiangdu@tlu.edu.cn

Qingchen Wang
School of electrical engineering
Tongling University
Tongling, China
2309171016@mail.tlu.edu.cn

Xinyu Zhang
School of electrical engineering
Tongling University
Tongling, China
2311151038@mail.tlu.edu.cn

Panpan Lu
School of electrical engineering
Tongling University
Tongling, China
2309171032@mail.tlu.edu.cn

***Abstract*—This paper addresses the challenge of balancing efficiency and resource consumption in the implementation of the SM4 algorithm. A high-performance, low-resource FPGA-based implementation scheme for the SM4 algorithm is proposed. The scheme achieves a high level of hardware resource intensification by reusing folded round functions. Through a time-interleaved mechanism, 32 rounds of iterations are compressed into 8 cycles, reducing register toggling activity by 50%. Compared with existing schemes, the proposed scheme increases data throughput by 17%, significantly reduces resource occupancy, and achieves a better balance between resource consumption and performance.**

***Keywords*—FPGA; SM4 algorithm; pipelined structure**

I. INTRODUCTION

With the rapid development of IoT terminal devices and ed ge computing, the efficient implementation of lightweight crypt ographic algorithms in resource-constrained environments has become a research hotspot in the field of information security. SM4 algorithm is widely used in smart terminals, industrial con trol, and other fields due to its 128 bit key strength and hardwar e-friendly characteristics. The SM4 algorithm can be deployed on various hardware platforms such as Central Processing Unit s (CPU) [1-3], Graphics Processing Units (GPU) [4-6], and FP GAs [7-9]. In the past three years, FPGAs have witnessed rapid development. Technologically, the manufacturing process has been continuously refined, and the integration level has been gr eatly improved. The application fields have continued to expan d, playing an important role in data centers and AI acceleration. [10] However, traditional FPGA-based SM4 implementation s chemes often sacrifice power consumption for high throughput, making it difficult to meet the urgent demand for "energy-effic ient secure computing" in edge devices.

The circular architecture of the SM4 algorithm completes 32 rounds of operations using a single round operation module. Characterized by low resource consumption and low throughput, this architecture is well-suited for scenarios where hardware resources are limited. Early research employed pure sequential logic, controlling round iteration through a state machine, with the 32 round computation demanding 32 cycles to finish. Gao et al. introduced a feedback circular architecture. In this scheme, a Moore-type state machine was utilized to control the overall operational flow of the system, iteratively transforming data. Precomputed round keys were stored in registers, which effectively reduced the time and computational resources required for key expansion. However, this approach still suffered from low throughput [11]. On the other hand, the pipeline architecture breaks down the 32 - round computation into multiple pipeline stages. Each stage handles a single or partial round, executing different rounds in parallel. This design significantly enhances encryption efficiency and throughput, making it highly suitable for encryption application scenarios involving large amounts of data. Guan et al. halved the size of the pipeline in a 32 - stage pipeline architecture, completing 32 rounds of operations with two iterations of data. Although a throughput of 1.2 Gb/s was achieved, there was a substantial increase in resource occupation [12]. A comparative analysis of iterative and pipeline architectures has been carried out. The advantages and disadvantages of these two architectural designs are presented and contrasted in Table I

TABLE I. A COMPARATIVE ANALYSIS OF CIRCULAR ARCHITECTURE AND PIPELINE ARCHITECTURE

Type	Advantages	Disadvantages
Circular Architecture	Low resource consum ption	Low performance
Pipeline Architecture	High performance	High resource consum ption

From the aforementioned comparison, it is evident that the implementation of the SM4 algorithm on FPGA still presents certain limitations, posing challenges in achieving an optimal balance between efficiency and resource consumption. To address this issue, this paper introduces a hybrid architecture-based FPGA design scheme for the SM4 algorithm. By employing a folded round function reuse structure, the 32 rounds of iteration are condensed into just 8 cycles through a time-interleaved mechanism, effectively reducing register toggling activity by 50%. Experimental results demonstrate that the

979-8-3503-5758-5/25/$31.00 ©2025 IEEE

proposed scheme enhances data throughput by 17% while decreasing resource utilization to approximately one-fourth of the original requirement. This design achieves an effective trade-off between power consumption and performance, making it particularly well-suited for power-sensitive applications such as smart cards and IoT sensors.

II. THEORETICAL BASIS

The SM4 cryptographic algorithm is a type of block cipher. In this algorithm, both the data block length and the key length are 128 bits. And both the encryption algorithm and the key expansion algorithm employ a 32 round nonlinear iterative structure.

A. Basic Components of SM4 Algorithm

1) S-Box Substitution

S-Box substitution is the nonlinear transformation component in the SM4 cryptographic algorithm, which is a key part of the SM4 cryptographic algorithm. The S-Box substitution function performs nonlinear substitution on the bytes of the input data, increasing the complexity and unpredictability of the data. Assuming the input byte is a and the output byte is b, then the operation of the S-Box can be expressed as b = SBox(a).

2) Nonlinear Transformation τ

The nonlinear transformation of the SM4 algorithm (denoted as τ transformation) is a nonlinear substitution transformation with a word (32 bits) as the unit. Assuming the input word is A = (a_0, a_1, a_2, a_3), and the output word after S-Box substitution is B = (b_0, b_1, b_2, b_3), then:

$$\boldsymbol{B = \tau(A) = (SBox(a_0), SBox(a_1), SBox(a_2), SBox(a_3))} \quad (1)$$

3) Linear Transformation L

The linear transformation component L consists of two operations: circular shift and XOR. Both its input and output are 32 bits. Assuming the input of L is B and the word is C, then:

$$C = L(B) = B \oplus (B <<< 2) \oplus (B <<< 10) \oplus (B <<< 18) \oplus (B <<< 24) \quad (2)$$

4) Composite Transformation T

The composite transformation T is composed of the nonlinear transformation τ and the linear transformation L. Assuming the input of the composite transformation is X, First, apply the τ transformation to X, followed by the L transformation. The formula is presented below:

$$\boldsymbol{T(X) = L(\tau(X))} \quad (3)$$

5) Round Function F

The input of the round function F is (X0, X1, X2, X3), which are 32-bit words. The round key RK is also a 32-bit word. The operation of the round function F is as follows:

$$\boldsymbol{F(X_0, X_1, X_2, X_3, RK) = X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus RK)} \quad (4)$$

B. Key Expansion Algorithm

The key expansion, as well as the encryption and decryption algorithms of the SM4 lgorithm, are all structured as 32 round iterative processes.Each round of encryption uses a 32 bit round key. The round key is obtained from the master key through key expansion. Assuming the user key is MK (MK_0, MK_1, MK_2, MK_3), the output round key is RK_i, i = 0, 1, ..., 30, 31, and the intermediate data is K_i, i = 0, 1, ..., 34, 35, then the key expansion algorithm can be described as follows:

$$\boldsymbol{K = K_i \oplus T'(K_{i+1}, K_{i+2}, K_{i+3}, CK_i)} \quad (5)$$

$$\boldsymbol{(K_0, K_1, K_2, K_3) = (MK_0 \oplus FK_0, MK_1 \oplus FK_1, MK_2 \oplus FK_2, MK_3 \oplus FK_3)} \quad (6)$$

The T' transformation is basically the same as T in the round encryption function of the encryption algorithm, except that the linear transformation L is modified to the following L'

$$L'(B) = B \oplus (B <<< 13) \oplus (B <<< 23) \quad (7)$$

C. SM4 Encryption and Decryption Process

The SM4 algorithm inputs a 128-bit plaintext group during the encryption process and undergoes 32 rounds of round function processing. Each round operation includes XOR with the round key, non-linear transformation of the S-box, linear transformation L, and circular shift, and finally outputs the ciphertext. The decryption process is completely symmetrical to the encryption process.

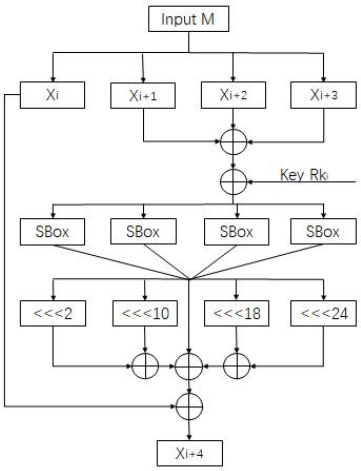

Fig. 1. Structure Diagram of SM4 Algorithm

The SM4 encryption algorithm adopts a 32-round iterative structure, using one round key for each round. Let the input plaintext be (X_0, X_1, X_2, X_3), a total of 4 words, 128 bits. The input round keys are RK_i, i=0,1,…,31, a total of 32 words. The output ciphertext is (Y_0, Y_1, Y_2, Y_3), a total of 4 words, 128 bits. The encryption algorithm can be described as $X_{i+4}=F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, RK_i)=X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus RK_i)$,i=0,1,…,31. After that, complete the reverse order transformation R: R(Y_0, Y_1, Y_2, Y_3)=(X_{35}, X_{34}, X_{33}, X_{32}) to obtain the ciphertext. The structure of the SM4 algorithm is shown in Figure 1.

III. THE IMPROVED ARCHITECTURE

Previous FPGA-based implementations of the SM4 algorithm have struggled to achieve a balance between efficiency and hardware resource consumption. This design scheme adopts folded round function multiplexing and time interleaving mechanism to optimize power consumption while maintaining performance.

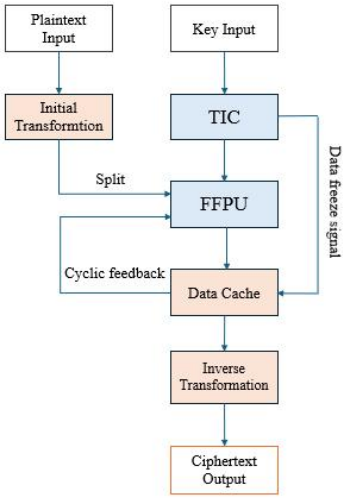

Fig. 2. Flowchart of Reusable Structure SM4 Algorithm

First, the plaintext is split into four-word data blocks through an initial transformation and input into a four-round folded processing unit; the time interleaving controller divides the 32 rounds of iteration into 8 time slices , with each slice dynamically loading four pre-extended round keys , activating the folded unit to perform round function calculations, generating intermediate data for caching and cyclic feedback; inactive time slices close the module clock through clock gating, while freezing register values to suppress power consumption; finally, the ciphertext is output through reverse order transformation. The flowchart is shown in Figure 2.

A. FFPU Module

Four-wheel folding processing module (FFPU) is the core computing module of this design. It compresses the four rounds of iteration of SM4 into single-cycle processing through a time-division multiplexing architecture.

The core structure includes parallel key mixing, a shared S-box array, and a reusable linear transformation module. Input data and round keys generate an intermediate state through a four-way XOR network, which is then subjected to nonlinear permutation by 4 composite field-optimized S-boxes and input into the linear transformation module to perform the standard linear transformation L of SM4. This module adopts a two-stage pipeline to realize cyclic shifting and XOR chains, which can optimize the delay of the critical path. Output registers suppress flipping in inactive cycles through clock gating (ClkEn) and data freezing technology to reduce dynamic power consumption. As shown in Figure 3.

B. TIC Module

Time-Interleaved Controller (TIC) is the scheduling core of the folded SM4 encryption architecture, and its working principle is based on the cooperative mechanism of key expansion, phase slicing, and dynamic gating.

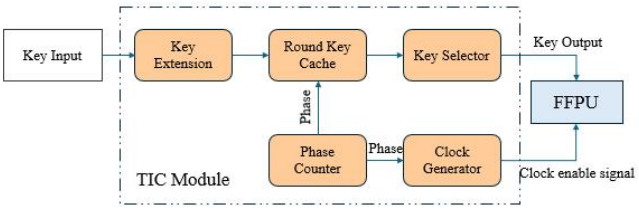

Fig 4. Schematic Diagram of TIC Module Structure

Firstly, the controller expands the original key into 32 round keys (Rk_0~Rk_{31}) through the SM4 standard algorithm and pre-stores them in the round key cache composed of dual-port RAM. Subsequently, the phase counter cyclically generates Phase signals with a period of eight, where each phase corresponds to four rounds of encryption operations. By calculating the address offset, round keys are selected in real-time and pushed to the folded processing unit. The clock gating module generates the ClkEn signal based on the phase state, activating the encryption circuit only during valid phase windows, while freezing the intermediate data registers during inactive cycles to suppress redundant toggling and reduce dynamic power consumption. The module structure is shown in Figure 4:

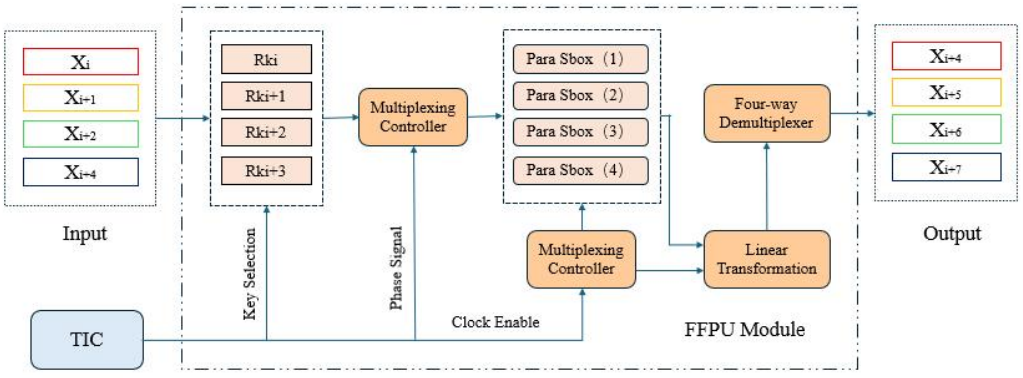

Fig 3. Schematic Diagram of FFPU Module

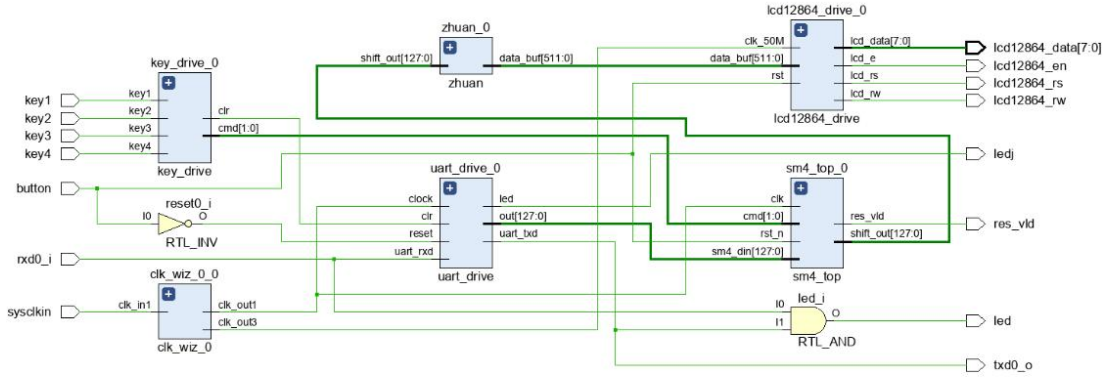

Fig 5. Top-Level Architecture RTL Diagram

IV. SIMULATION RESULT

A. *Top-Level Architecture Design*

The FPGA verification in this study is based on the xc7a35tfgg484 chip from Xilinx. The design was developed using the Vivado 2020.2 toolchain, mapping the folded round function architecture to synthesizable Verilog modules at the hardware description language level. As is shown in Figure 5.The core encryption module contains four parallel processing channels, each integrating configurable 4-round folded units, achieving S-box reuse and linear transformation hardware sharing through parametric design. The Time-Interleaved Controller module implements eight-phase cyclic scheduling through a state machine, dynamically loading four sets of precomputed round keys in each clock cycle while generating clock enable signals to control the gated clock network of the encryption unit.

B. *Result Testing and Analysis*

Based on the xc7a35tfgg484 chip, SM4 algorithm proposed in this paper was synthesized and implemented. The implementation results consumed 969 LUTs and 910 FFs, with a maximum operating frequency of 280 MHz and a throughput rate of 0.89 Gbps. To better analyze the performance of this scheme, a comparison was made with FPGA implementation schemes of the SM4 algorithm in some literature, as shown in Table Ⅱ:

TABLE II. SCHEME COMPARISON ANALYSIS

Design	Resource	Maximum Operating Frequency (MHz)	Throughput (Gb/s)
Work in [11]Balance-8	2949LUT 2753FF	198	0.63
Work in [11]Balance-16	4506LUT 4033FF	191	1.2
Work in [13]	1796LUT 301FF	270	0.76
Our work	969LUT 910FF	280	0.89

From the data in the table, compared with the Balance-8 and Balance-16 schemes in Literature 11, under similar throughput performance, the resource consumption is about 1/4. Compared with the scheme in Literature 15 (Note: Literature 15 is not listed in the provided text, so it's assumed as a reference mentioned elsewhere), with similar resource occupancy, the maximum operating frequency is increased, and the data throughput is improved by about 17%. Overall, the scheme in this paper achieves a good balance between logic resource consumption and performance, with low logic resource consumption and significant improvements in both system frequency and data throughput rate.

V. CONCLUSION

Aiming at the current issues of low implementation efficiency and high resource consumption in SM4 algorithm, this paper proposes a high-performance, low-resource-consumption FPGA-based SM4 algorithm implementation scheme. This scheme achieves a high degree of intensification of hardware resources through folded round function reuse and compresses 32 rounds of iteration into 8 cycles through a time-interleaved mechanism, reducing register toggling activity by 50%. Compared with existing schemes, this scheme improves data throughput by 17%, significantly reduces resource occupancy, and demonstrates advanced performance.

ACKNOWLEDGMENT

This work was financially supported by Natural Science Research Project of Tongling University(2022tlxy41) and College Students' Innovation and Entrepreneurship Projects (S202410383023).

REFERENCES

[1] C Chen, H Guo, Y Liu, et al, "Register-based SM4 software optimization implementation method," Journal of Cryptographic Research , 427-440, 2024.

[2] C Chen, H Guo, CH Wang, et al. "A fast software implementation method for the national cryptographic SM4 algorithm based on composite fields," Journal of Cryptographic Research, 10(2): 289-305, 2023.

[3] L Wang, Z Gong, Z Liu, et al. "Fast software implementation of SM4 algorithm based on tower fields," Journal of Cryptographic Research, 9(6): 1081-1098, 2022.

[4] J Dong, Y Huang, Y Fu, et al. "Research progress on high-performance cryptographic computing technology based on heterogeneous multi-core GPUs," Journal of Software, 1-27, 2024.

[5] X Li, Y Hu, Y Chi, et al. "Parallel implementation and optimization of SM4-CTR algorithm based on general computing platforms," Journal of Cryptographic Research, 9(4): 663-676,2022.

[6] X Li, C Ji, X Duan, et al. "Parallel implementation of SM4 algorithm on GPU ," Information Network Security, 20(6): 36-43,2020.

[7] Q Zhu, X Chen, J Lu. "Hardware design and FPGA implementation of variable pipeline stage SM4 encryption and decryption algorithm," Computer Engineering and Science, 46(4): 606-614,2024.

[8] J Zhai, B Li, Q Zhou, et al. "FPGA-Based High-Performance and Scalable SM4-GCM Algorithm Implementation," Computer Science, 49(10): 74-82,2022.

[9] K Wang, K Liu, T Li, et al. "Design and Implementation of a Reconfigurable High-Speed Data Encryption System," Electronic Measurement Technology, 44(19): 8-15,2021.

[10] C Pham-Quoc and T Ngoc Thinh, "Efficient FPGA-Based Convolutional Neural Network Implementation for Edge Computing," Journal of Advances in Information Technology, 14(3) : 479-487, 2023.

[11] Gao X W, Lu E H, Xian L Q, et al. "FPGA Implementation of the SMS4 Block Cipher in the Chinese WAPI Standard," IEEE Press, 104-106,2008.

[12] G Zheny, L Yunhao, T shang, et al. "Implementation of SM4 on FPGA: Trade-off Analysis Between Area and Speed," IEEE Press, 192-197,2018.

[13] H Nie, Y Han, K Li. "FPGA Implementation and Optimization of Agile Development for SM4 Algorithm," Integrated Circuits and Embedded Systems, 24(7): 80-84,2024.